

Vite & Gourmand

Documentation technique de l'application web

ECF — Titre professionnel Développeur Web et Web Mobile

Auteur : Boris NADAL (FastDev)

Version 1.0 — septembre 2026

Sommaire

1. Réflexions initiales technologiques
2. Configuration de l'environnement de travail
3. Architecture de l'application
4. Modèle conceptuel de données et base NoSQL
5. Diagramme de cas d'utilisation
6. Diagrammes de séquence, d'états et de classes
7. Règles de gestion implémentées
8. Sécurité
9. Accessibilité (RGAA) et RGPD
10. Tests
11. Documentation du déploiement

1. Réflexions initiales technologiques

1.1 Analyse du besoin

Vite & Gourmand est une petite entreprise (deux gérants) qui souhaite présenter ses menus et prendre des commandes en ligne. Les besoins structurants qui ont guidé mes choix sont :

- un **catalogue relationnel** fort (menus ↔ plats ↔ allergènes, thèmes, régimes, commandes, avis) : une base relationnelle est indispensable ;
- quatre profils (visiteur, utilisateur, employé, administrateur) avec des droits différents : un **contrôle d'accès par rôle** côté serveur ;
- des interactions dynamiques sans rechargement (filtres, calcul du prix) : du **JavaScript** côté client et une petite **API JSON** ;
- des statistiques pour l'administrateur imposées depuis une **base non relationnelle** ;
- un budget et une équipe réduits : une solution simple à maintenir, peu coûteuse à héberger, sans dépendances lourdes.

1.2 Choix retenus et justification

Couche	Choix	Justification	Alternatives étudiées
Front-end	HTML5, CSS3, Bootstrap 5.3, JavaScript natif (fetch), Chart.js 4	Bootstrap fournit une grille responsive et des composants accessibles (menu burger, carrousel, alertes) ; je le personnalise avec la charte. Le JavaScript natif suffit pour les filtres et le calcul du prix : pas d'étape de build, pas de framework à maintenir. Chart.js pour le graphique administrateur.	React / Vue.js : pertinents pour une application très interactive, mais surdimensionnés ici (build, SEO, double logique de routage).

Back-end	PHP 8.3 sans framework, architecture MVC, PDO	PHP est hébergeable partout et c'est la technologie étudiée en formation. Coder le socle (routeur, CSRF, validation, authentification) m'oblige à comprendre et à maîtriser chaque mécanisme de sécurité. <code>declare(strict_types=1)</code> et le typage PHP 8 fiabilisent le code.	Symfony : très complet (ORM, formulaires, sécurité), mais courbe d'apprentissage et poids importants pour ce périmètre. Node.js/Express : cohérent avec JS, mais PHP était imposé en exemple par la stack.
Base relationnelle	MySQL 8 / MariaDB 10.11	Répond au MCD fourni, gère les transactions (InnoDB), les clés étrangères, les contraintes <code>CHECK</code> . Largement disponible (offre gratuite Aiven en production).	PostgreSQL : excellent, mais MySQL est plus répandu chez les hébergeurs PHP.
Base NoSQL	MongoDB (extension officielle <code>ext-mongodb</code>)	Chaque commande est dupliquée sous forme de document dénormalisé ; les agrégations (<code>\$match</code> , <code>\$group</code>) calculent le nombre de commandes et le chiffre d'affaires par menu sans jointure et sans charger la base métier. Offre gratuite MongoDB Atlas.	Redis : orienté cache clé/valeur, moins adapté aux agrégations.
E-mails	Client SMTP interne + Brevo	Pas de dépendance externe ; un pilote « log » écrit les e-mails dans un fichier en développement.	PHPMailer (via Composer).
Géolocalisation	API Adresse (data.gouv.fr) + OSRM	Gratuits, sans clé : calcul de la distance routière pour la livraison hors Bordeaux, avec repli sur une estimation (formule de haversine) ou une saisie manuelle.	Google Maps API : payante, clé obligatoire.
Déploiement	Docker + Fly.io, Aiven (MySQL), MongoDB Atlas	L'image Docker garantit le même environnement en local et en production. Fly.io déploie un conteneur en région Paris et fournit HTTPS automatiquement.	Heroku (plus d'offre gratuite), Vercel (orienté Node/serverless, pas adapté à PHP + sessions).

2. Configuration de l'environnement de travail

Outil	Usage et configuration
Visual Studio Code	Éditeur. Extensions : PHP Intelephense (autocomplétion, analyse), PHP Debug (Xdebug), EditorConfig, Prettier (JS/CSS), GitLens, MongoDB for VS Code.
PHP 8.3 (CLI)	Extensions <code>pdo_mysql</code> , <code>mbstring</code> , <code>fileinfo</code> , <code>mongodb</code> (installée avec <code>pecl install mongodb</code>). Serveur de développement intégré : <code>php -S localhost:8000 -t public public/router.php</code> .
MariaDB 10.11 / MySQL 8	Base locale ; création d'un utilisateur applicatif <code>vg_app</code> limité à SELECT/INSERT/UPDATE/DELETE (moindre privilège). Client : MySQL Workbench / ligne de commande.
MongoDB 7 + Compass	Base locale pour les statistiques, visualisation de la collection <code>commandes_stats</code> .
Docker Desktop	<code>docker compose up</code> lance l'application, MariaDB (import automatique des scripts SQL) et MongoDB : même environnement que la production.
Git + GitHub	Branches <code>main</code> / <code>develop</code> / <code>feature/*</code> , dépôt public.

Navigateurs	Chrome et Firefox (outils de développement, mode responsive, onglet Lighthouse pour l'accessibilité et les performances).
Playwright	Tests de bout en bout automatisés des parcours (voir §10).
Notion	Gestion de projet (tableau Kanban), voir la documentation de gestion de projet.
Maquettes	Wireframes basse fidélité en HTML/CSS, mockups haute fidélité (voir la charte graphique).

La configuration est externalisée dans un fichier `.env` (non versionné) construit à partir de `.env.example` : identifiants de base de données, URI MongoDB, paramètres SMTP, mode debug. Les secrets ne sont jamais commités.

3. Architecture de l'application

L'application suit le patron **MVC** avec un **contrôleur frontal** unique (`public/index.php`). Seul le dossier `public/` est exposé par le serveur web : le code source, les scripts SQL et le fichier `.env` ne sont pas accessibles depuis Internet.

```
public/          index.php (contrôleur frontal + routes), assets/ (css, js, img,
polices, vendor), uploads/
src/Core/        Router, Database (PDO), Mongo, Auth, Csrf, Validator, Mailer, Upload,
Env
src/Controllers/ Home, Menu, Auth, Commande, Compte, Contact, Employe, Admin
src/Models/      Model (base), Utilisateur, Menu, Plat, Catalogue, Commande, Avis,
Tarif (règles de prix)
views/           layout, pages, compte, employe, admin, partials, emails
database/        00_database.sql, 01_schema.sql, 02_data.sql
bin/             sync_mongo.php (reconstruction de la collection MongoDB)
```

Cycle d'une requête : `index.php` démarre une session sécurisée et envoie les en-têtes de sécurité → le `Router` trouve la route → il vérifie le rôle requis (`Auth::requireRole`) puis le jeton CSRF pour tout POST → le contrôleur valide les données (`Validator`), appelle les modèles (requêtes préparées PDO) → la vue est rendue dans le gabarit commun, toutes les sorties étant échappées par `e()`.

Route	Accès	Rôle
/, /menus, /menus/{id}, /api/menus, /contact, /cgv, /mentions-legales, /confidentialite	Public	Consultation, filtres (JSON)
/inscription, /connexion, /mot-de-passe-oublie, /reinitialiser-mot-de-passe	Public	Authentification
/commande, /api/prix, /compte/...	Connecté	Commande, suivi, avis, profil
/employe/...	Employé, admin	Commandes, menus, plats, horaires, avis
/admin, /admin/employes, /api/admin/stats	Admin	Comptes employés, statistiques MongoDB

4. Modèle conceptuel de données et base NoSQL

Le modèle reprend le MCD de l'annexe 1 et le complète pour couvrir l'ensemble du cahier des charges :

- **menu_image** : galerie d'images d'un menu (avec texte alternatif, RGAA) ;
- **commande_suivi** : historique horodaté de chaque changement d'état (exigé pour le suivi client) ;
- dans **commande** : adresse, distance, réduction, instantané des coordonnées du client, motif d'annulation et mode de contact ;
- **reinitialisation_mdp** (jetons hachés) et **tentative_connexion** (anti brute-force) pour la sécurité ;
- **message_contact** : conservation des demandes en plus de l'e-mail ;
- **actif** dans utilisateur (désactivation d'un employé) et dans menu (archivage d'un menu déjà commandé).

Les associations n,n *propose* (menu-plat) et *contient* (plat-allergène) deviennent les tables **menu_plat** et **plat_allergene** ; les associations 1,1 *propose* (thème) et *adapte* (régime) deviennent des clés étrangères dans **menu**.

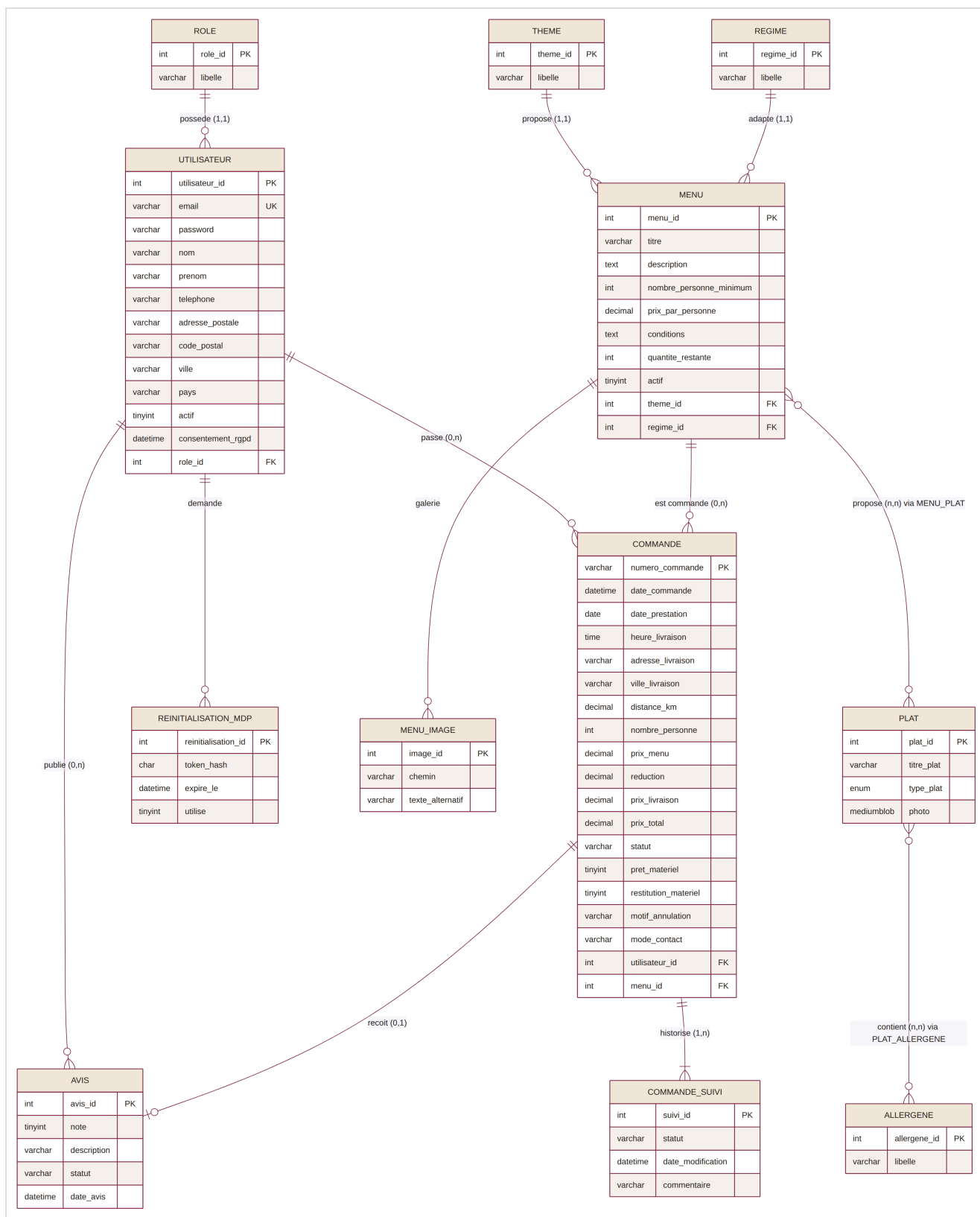


Figure 1 — Modèle de données (notation « pattes de corbeau », cardinalités indiquées sur les associations)

Structure du document MongoDB (collection `commandes_stats`)

```
{
  "_id": "VG-20260922-A1B2C3",          // numéro de commande (clé partagée avec MySQL)
  "menu_id": 5, "menu_titre": "Bistrot Bordelais",
  "nombre_personne": 15, "prix_menu": 432.00, "prix_livraison": 12.08, "prix_total":
444.08,
  "statut": "en_attente",
  "date_commande": ISODate("2026-09-22T10:15:00Z")
}
```

Le document est inséré (upsert) à la création, mis à jour à chaque changement de statut, et la collection peut être reconstruite à tout moment avec `php bin/sync_mongo.php`. L'agrégation utilisée par l'administrateur :

```
[ { $match: { statut: { $ne: "annulee" }, menu_id: 5, date_commande: { $gte: ..., $lte: ... }
} },
  { $group: { _id: "$menu_titre", nombre_commandes: { $sum: 1 }, chiffre_affaires: {
$sum: "$prix_total" } } },
  { $sort: { nombre_commandes: -1 } } ]
```

5. Diagramme de cas d'utilisation

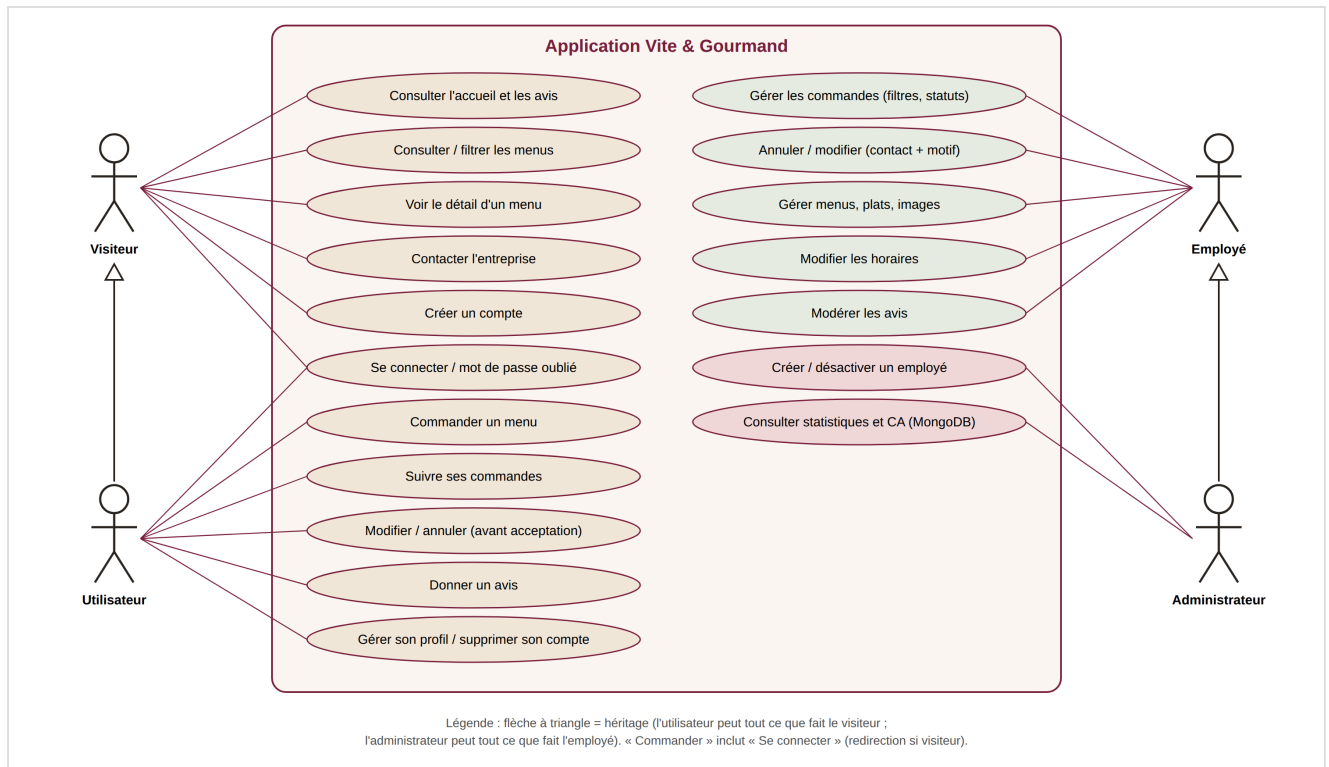


Figure 2 — Cas d'utilisation par acteur

6. Diagrammes de séquence, d'états et de classes

6.1 Commande d'un menu

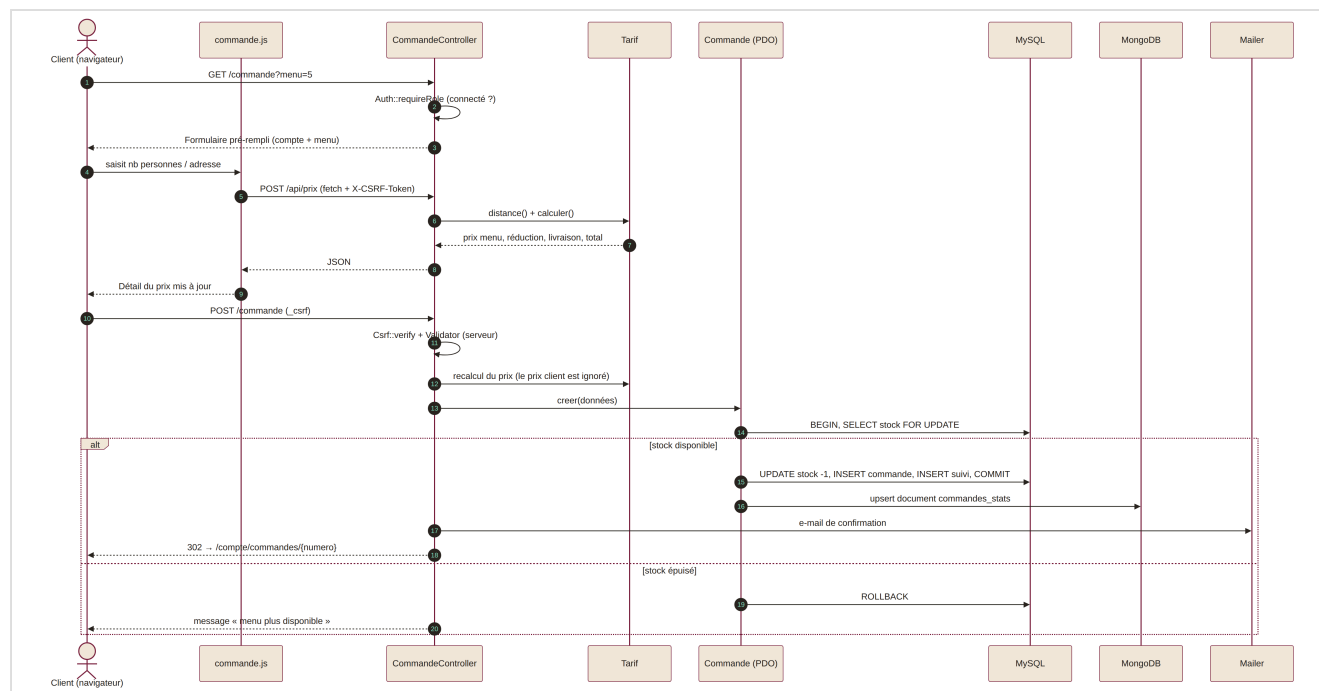


Figure 3 — Séquence « commander un menu » (prix recalculé côté serveur, stock verrouillé en transaction)

6.2 Connexion

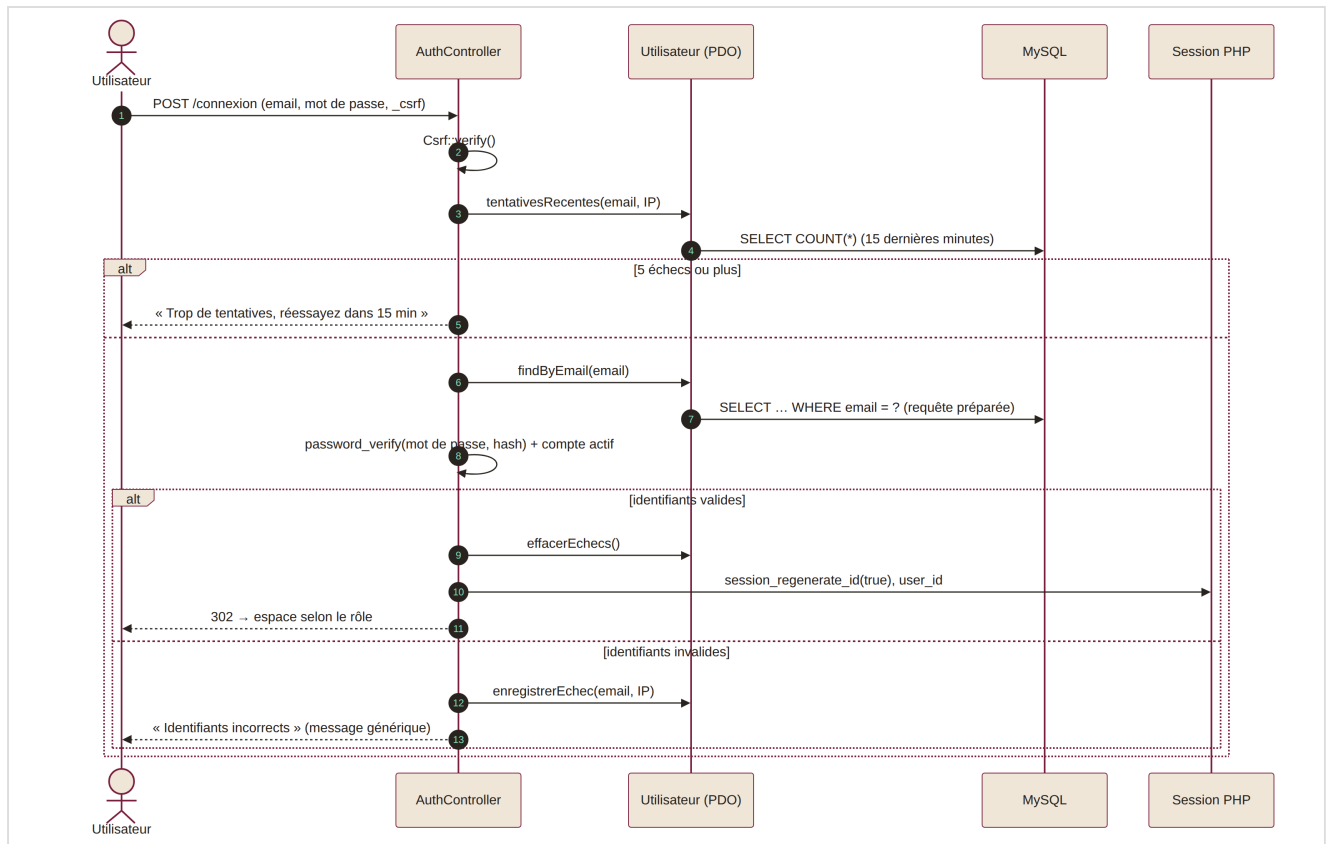


Figure 4 — Séquence « se connecter » avec limitation des tentatives

6.3 Mise à jour du statut par un employé

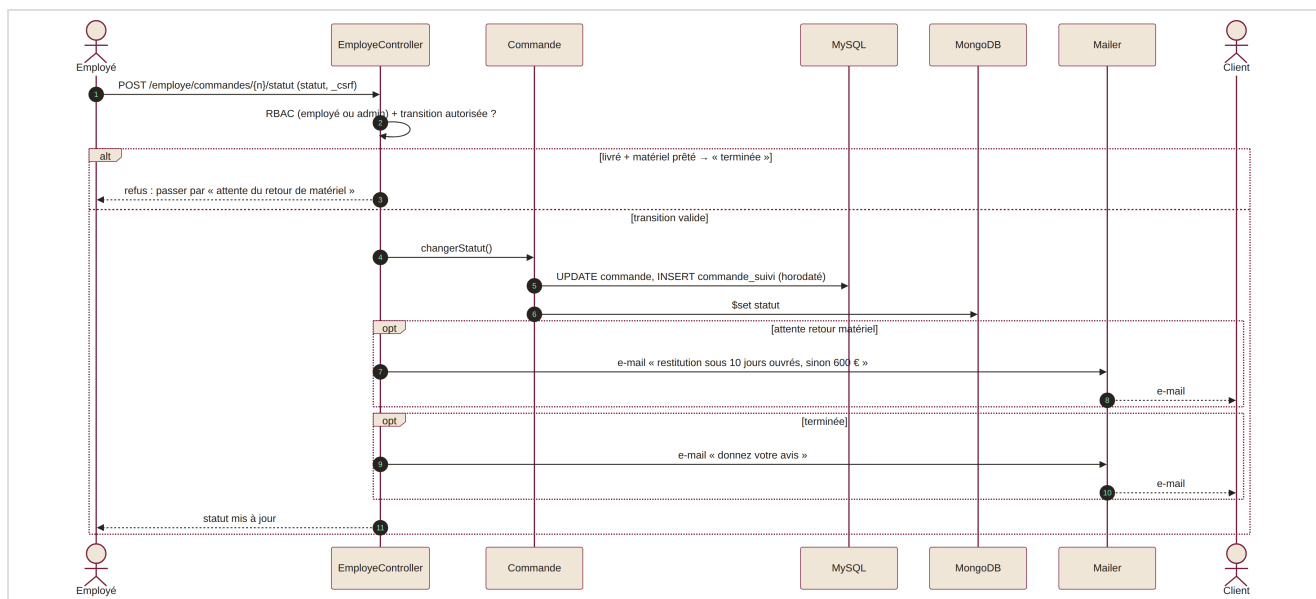


Figure 5 — Séquence « changer le statut d'une commande »

6.4 Cycle de vie d'une commande

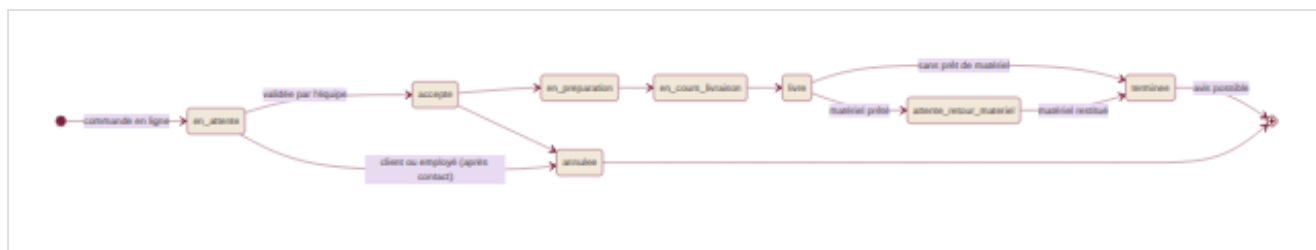


Figure 6 — Diagramme d'états (machine à états codée dans `Commande::TRANSITIONS`)

6.5 Diagramme de classes (principales)

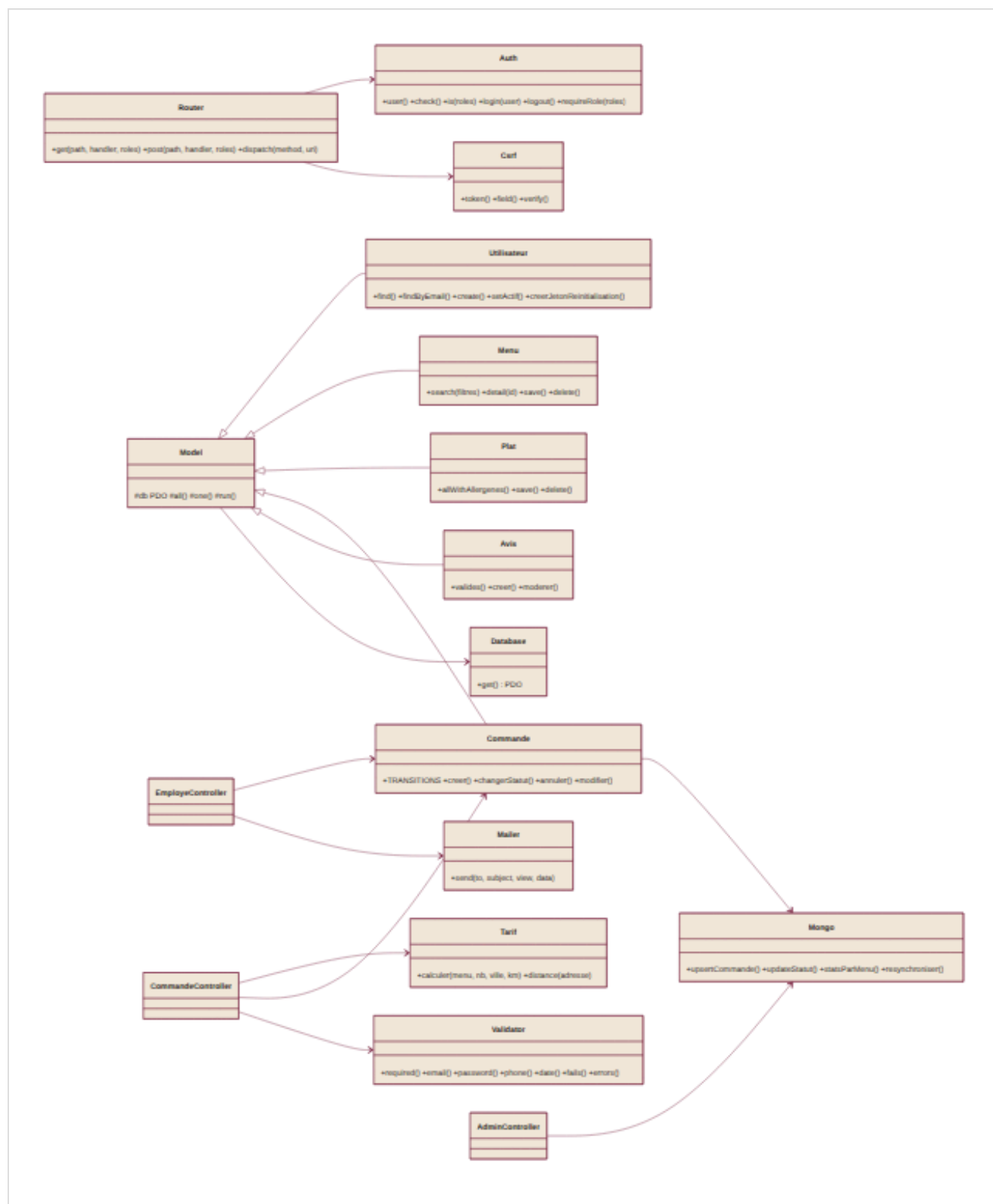


Figure 7 — Classes du socle, des modèles et des contrôleurs

7. Règles de gestion implémentées

Règle	Implémentation
Nombre minimum de personnes	Validation serveur (<code>intBetween</code>) + attribut <code>min</code> côté client.
Réduction de 10 % à partir de min + 5 personnes	<code>Tarif::calculer()</code> , affichée dans le détail du prix.
Livraison : gratuite à Bordeaux, sinon 5 € + 0,59 €/km	<code>Tarif::distance()</code> (API Adresse + OSRM, repli haversine × 1,3 ou saisie).
Prix détaillé avant validation	API <code>POST /api/prix</code> appelée par <code>commande.js</code> ; le serveur recalcule toujours à l'enregistrement.
Stock disponible	Décrément en transaction avec <code>SELECT ... FOR UPDATE</code> ; rendu en cas d'annulation ; menu « épuisé » à 0.
Modification / annulation client	Uniquement au statut « en attente » ; le menu n'est pas modifiable.
Annulation / modification employé	Mode de contact (GSM / mail) et motif obligatoires, tracés dans le suivi.
Prêt de matériel	« Livré » → « en attente du retour de matériel » obligatoire, e-mail (10 jours ouvrés, 600 €).
Avis	Possible quand la commande est « terminée » (e-mail d'invitation), un seul avis par commande, publié après validation.
Comptes	Rôle « utilisateur » à l'inscription ; comptes employés créés par l'admin (mot de passe non envoyé) ; aucun admin créable depuis l'application.
Suppression d'un menu déjà commandé	Archivage (<code>actif = 0</code>) pour conserver l'historique.

8. Sécurité

Menace (OWASP Top 10:2025)	Mesures mises en place
A05 Injection (SQL)	PDO avec requêtes préparées natives (<code>ATTR_EMULATE_PREPARES = false</code>) partout ; aucune valeur concaténée ; filtres typés (cast int/float) ; <code>LIKE</code> échappé.
Injection (XSS)	Échappement de toutes les sorties (<code>htmlspecialchars</code> via <code>e()</code>) ; <code>escapeHtml()</code> côté JS avant insertion dans le DOM ; Content-Security-Policy stricte (<code>script-src 'self'</code> , aucun script en ligne).
CSRF	Jeton aléatoire de 256 bits par session, vérifié par <code>hash_equals</code> sur tout POST (champ caché ou en-tête <code>X-CSRF-Token</code> pour fetch) ; cookie <code>SameSite=Lax</code> .
A01 Contrôle d'accès	Rôle vérifié dans le routeur ; rôle relu en base à chaque requête ; contrôle d'appartenance des commandes (anti-IDOR) ; rôle « employé » forcé côté serveur ; pas de création d'admin.

A07 Authentification	<code>password_hash</code> / <code>password_verify</code> (bcrypt, re-hachage automatique) ; politique 10 caractères + majuscule, minuscule, chiffre, spécial ; blocage 15 min après 5 échecs (e-mail + IP) ; message d'erreur générique ; <code>session_regenerate_id</code> à la connexion ; expiration après 30 min d'inactivité.
Réinitialisation du mot de passe	Jeton aléatoire de 256 bits, seul son hash SHA-256 est stocké, valable 1 h, à usage unique ; même message que le compte existe ou non.
A02 Mauvaise configuration	Erreurs masquées en production et journalisées ; en-têtes <code>X-Frame-Options</code> , <code>X-Content-Type-Options</code> , <code>Referrer-Policy</code> , <code>HSTS</code> ; cookies <code>HttpOnly</code> / <code>Secure</code> ; <code>expose_php=Off</code> ; seul <code>public</code> / exposé.
Téléversement de fichiers	Type MIME réel (finfo) + <code>getimagesize</code> , 2 Mo max, nom aléatoire, exécution PHP interdite dans <code>uploads/</code> .
A04 Défaillances cryptographiques	HTTPS forcé (Fly.io), connexion TLS à la base Aiven, secrets dans des variables d'environnement.
A03 Chaîne d'approvisionnement	Aucune dépendance Composer/npm en production ; Bootstrap et Chart.js servis localement (pas de CDN tiers).
A10 Conditions exceptionnelles	Transactions avec <code>rollback</code> ; gestionnaire global d'exceptions ; échec MongoDB ou SMTP journalisé sans bloquer la commande.
Autres	Validation serveur de toutes les saisies ; protection contre l'injection d'en-têtes e-mail ; redirections internes uniquement ; pot de miel anti-robot sur le contact ; utilisateur SQL aux droits limités.

9. Accessibilité (RGAA) et RGPD

RGAA 4.1 : langue déclarée (`lang="fr"`), lien d'évitement, structure de titres hiérarchique et balises sémantiques (`header` , `nav` , `main` , `footer`), étiquettes `label` sur tous les champs, champs obligatoires signalés, erreurs reliées aux champs (`aria-describedby` , `aria-invalid`) et résumé des erreurs recevant le focus, résultats des filtres annoncés (`aria-live`), focus visible, contrastes AA vérifiés (texte 14,2:1, boutons 10:1), alternatives textuelles des images, tableaux avec `caption` et `scope` , navigation complète au clavier, respect de `prefers-reduced-motion` .

RGPD : consentement explicite à l'inscription (horodaté), politique de confidentialité, minimisation des données, droit d'effacement (suppression du compte depuis l'espace, commandes anonymisées), un seul cookie strictement nécessaire (pas de bandeau requis), polices auto-hébergées (aucune IP transmise à Google), durée de conservation définie.

10. Tests

Un scénario Playwright automatisé rejoue **57 vérifications** sur l'application réelle (MariaDB + MongoDB), toutes réussies : filtres dynamiques, inscription (mot de passe faible refusé), e-mails, calcul du prix (444,08 € pour 15 personnes à 12 km : 480 – 48 + 5 + 7,08), minimum de personnes, modification/annulation, IDOR, rôles (403), CSRF (419), cycle des statuts, prêt de matériel, horaires, CRUD menu avec image, refus d'un faux PNG contenant du PHP, modération, avis, statistiques MongoDB et filtres, création/désactivation d'employé, réinitialisation du mot de passe à usage unique, blocage après 5 échecs, menu mobile.

11. Documentation du déploiement

11.1 Démarche

L'application est conteneurisée (Dockerfile : PHP 8.3 + Apache, extensions `pdo_mysql` et `mongodb`, racine web sur `public/`, port 8080). Les données sont hébergées par des services managés gratuits, ce qui évite de maintenir des serveurs de base de données :

Composant	Service	Remarque
Application PHP	Fly.io (région <code>cdg</code> Paris)	HTTPS automatique ; machine shared-cpu-1x 256 Mo ($\approx$ 2 \$/mois, carte bancaire requise).
MySQL	Aiven for MySQL (offre gratuite)	Connexion TLS avec le certificat CA d'Aiven.
MongoDB	MongoDB Atlas M0 (gratuit)	Chaîne <code>mongodb+srv://</code> .
E-mails	Brevo (SMTP, offre gratuite)	<code>smtp-relay.brevo.com:587</code> , STARTTLS.

11.2 Étapes

1. **Base MySQL** : créer le service Aiven for MySQL, récupérer hôte, port, utilisateur `avnadmin`, mot de passe et télécharger `ca.pem`. Importer les scripts :

```
mysql --ssl-ca=ca.pem -h HOTE -P PORT -u avnadmin -p defaultdb <
database/01_schema.sql
mysql --ssl-ca=ca.pem -h HOTE -P PORT -u avnadmin -p defaultdb <
database/02_data.sql
```

2. **MongoDB Atlas** : créer un cluster M0, un utilisateur de base de données, autoriser l'accès réseau (`0.0.0.0/0`, les adresses IP de Fly.io n'étant pas fixes) et copier la chaîne de connexion.
3. **Brevo** : créer un compte, valider l'adresse expéditrice et générer une clé SMTP.
4. **Fly.io** : installer `flyctl`, puis `fly auth login` et `fly launch --no-deploy` (le fichier `fly.toml` du dépôt est réutilisé).
5. **Secrets** (jamais dans Git) :

```
fly secrets set APP_URL=https://vite-et-gourmand.fly.dev DB_HOST=... DB_PORT=... \
DB_NAME=defaultdb DB_USER=avnadmin DB_PASSWORD=... DB_SSL_CA_PEM="$(cat ca.pem)" \
MONGODB_URI="mongodb+srv://utilisateur:motdepasse@cluster0.xxxx.mongodb.net" \
SMTP_USER=... SMTP_PASSWORD=... MAIL_FROM=no-reply@... MAIL_CONTACT=...
```

6. **Déploiement** : `fly deploy` construit l'image et démarre la machine.
7. **Initialisation MongoDB** : `fly ssh console -C "php /var/www/html/bin/sync_mongo.php"`.
8. **Vérifications** : ouverture de l'URL en HTTPS, connexion avec chaque compte de démonstration, passage d'une commande, réception des e-mails, affichage des statistiques ; `fly logs` pour le suivi.

11.3 Mise à jour

Après fusion de `develop` dans `main` : `git pull` puis `fly deploy`. Les évolutions de schéma sont livrées sous forme de scripts SQL numérotés. Les images téléversées étant stockées sur le disque de la machine, un volume Fly (`fly volumes create`) peut être monté sur `public/uploads` pour les conserver entre deux déploiements.